Lab: RAG + Tools

Modulo 3 · Lezione 9



Quattro domande, dove sta la risposta

«Entro quanti giorni posso restituire un prodotto comprato online?»	documenti · la policy: 30 giorni
«Quanti ordini ha fatto Giulia Moretti?»	dati · il database: 13
«Lorenzo Rossi può montare il braccio LMX-4400 sulle scrivanie che ha ordinato?»	documenti+dati · gli ordini nel database, la compatibilità nel listino
«Che tempo fa oggi a Padova?»	da nessuna parte

il RAG del modulo (Modulo 3 · Lezione 2-6) · legge solo i documenti

«Quanti ordini ha fatto Giulia Moretti?»

Non lo trovo nei documenti.

il giro con query_db (Modulo 3 · Lezione 7-8) · legge solo i dati

«Entro quanti giorni posso restituire…?»

nessun tool legge la policy: risponde a memoria, senza poterla citare

> Ognuno fa una cosa sola. Oggi: **un assistente con tutti e due i tool**, che sceglie, e che quando nessuno dei due ha la risposta lo dice.



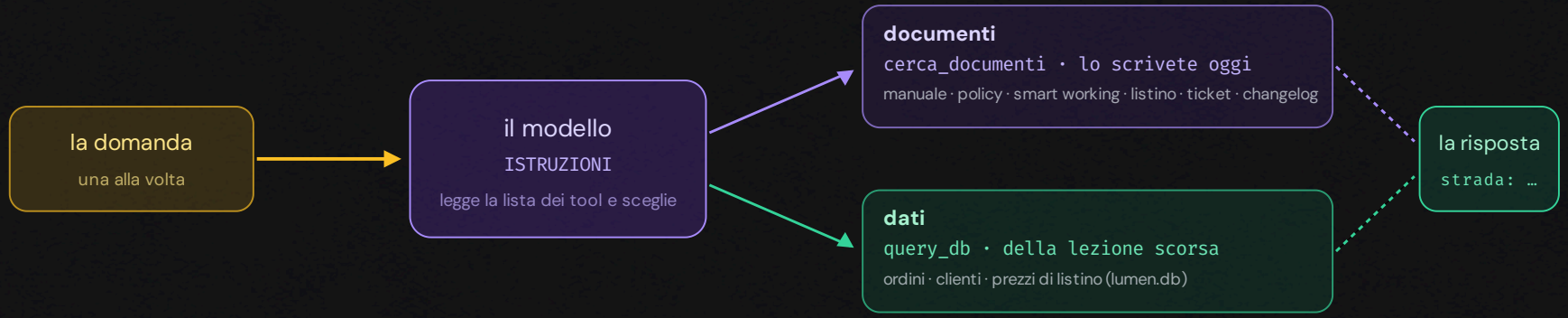
Agenda

	Blocco	Dove	Tempo
	Brief: le due fonti, il giro e il pezzo che scrivete, il RAG come tool, il rifiuto	slide	35′
	Setup: venv, chiave, primo lancio	`scripts/`	10′
	Lab 1a: attaccare un tool (TODO 1) · poi 5′ di rientro	`working/assistente.py`	45′ + 5′
	Lab 1b: il RAG come tool (TODO 2) · poi 5′ di rientro	`working/assistente.py`	40′ + 5′
	Pausa		15′
	Lab 1c: le istruzioni (TODO 3) · poi 5′ di rientro	`working/assistente.py`	25′ + 5′
	Lab 2: provare e sistemare · poi 10′ di rientro finale	`working/assistente.py` · `working/griglia.md`	45′ + 10′

Un file solo, `working/assistente.py`, nella cartella `scripts/` di questa lezione; la scheda del Lab 2 è `working/griglia.md`. Consegna in `CONSEGNA.md`.



Le due fonti e le strade

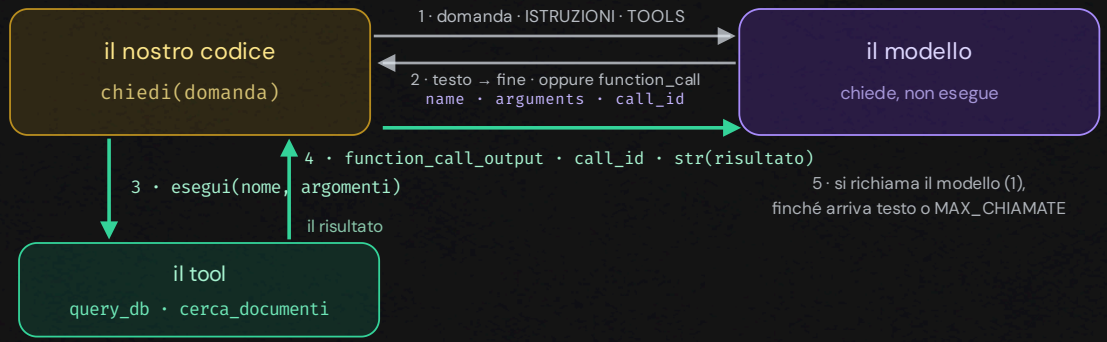


documenti	dati	documenti+dati	nessuna
ha usato cerca_documenti	ha usato query_db	due tool nello stesso giro	nessun tool: la risposta non sta da nessuna parte, e lo dice

strada attesa: dove sta la risposta

strada presa: quali tool ha chiamato il modello

Il giro, e il pezzo che scrivete oggi



3 e 4: oggi li scrivete voi

il giro di Modulo 3 · Lezione 7-8: allora l'avete letto, oggi lo scrivete

Attaccare un tool: tre punti nel vostro file.

`la voce in TOOLS`

`definisci_tool(schema, nome)`: dalla classe alla definizione che il modello legge. La docstring è la descrizione, i campi sono i parametri.

`il ramo in esegui()`

dal nome alla funzione, con gli argomenti giusti

`il ciclo, dentro chiedi() · le frecce 3 e 4`

`arguments` è una stringa JSON: `json.loads`. Il risultato torna come `function_call_output` con il `call_id` della richiesta, come stringa.

Il RAG diventa un tool: cerca_documenti

`domanda` → `rag.recupera(domanda, K)` → i K chunk, come testo:

```
[policy-resi-garanzia-001]
Hai 30 giorni di tempo dalla consegna per cambiare idea: il diritto di
recesso vale…
[policy-resi-garanzia-007]
Le regole cambiano per gli acquisti in promozione…
[…] altri due chunk
```

→ va al modello come `function_call_output`, come il risultato di una query.

~~`rag.rispondi() → la risposta già scritta`~~

(recupera + genera in una riga, Modulo 3 · Lezione 2-3) il modello non potrebbe **citare** i passaggi, né metterli insieme ai **dati** della domanda doppia

`rag.recupera() → i chunk con gli id`

il modello li legge, li cita, li combina

Le regole di citazione e di rifiuto le riscrivete voi nelle `ISTRUZIONI` dell'assistente (TODO 3): partite pure da quelle di `scaffolding/aikit/rag.py`.

Il retrieval restituisce **sempre** K chunk, anche fuori tema: se contengono la risposta lo decide il modello.



Come sceglie il modello: le due leve

Come alla lezione scorsa: legge tutto quello che riceve. Di ogni tool conosce nome, descrizione e parametri. La scelta si governa da **due posti**, e sono le due leve del Lab 2.

`1 · ISTRUZIONI`

quando usare quale tool, come citare, cosa fare se la risposta non c'è. Le scrivete nel TODO 3.

`2 · la docstring di CercaDocumenti, cioè la descrizione del tool · la scrivete voi nel TODO 2; questa è scarna apposta`

```
"""Cerca nei documenti di Lumen e ritorna i passaggi pertinenti, con il loro id."""
```

Nel Lab 2 si può arricchire: quali documenti ci sono, cosa non c'è. Serve quando il modello non entra nei documenti, o ci entra per i dati; per il resto la leva è la prima.

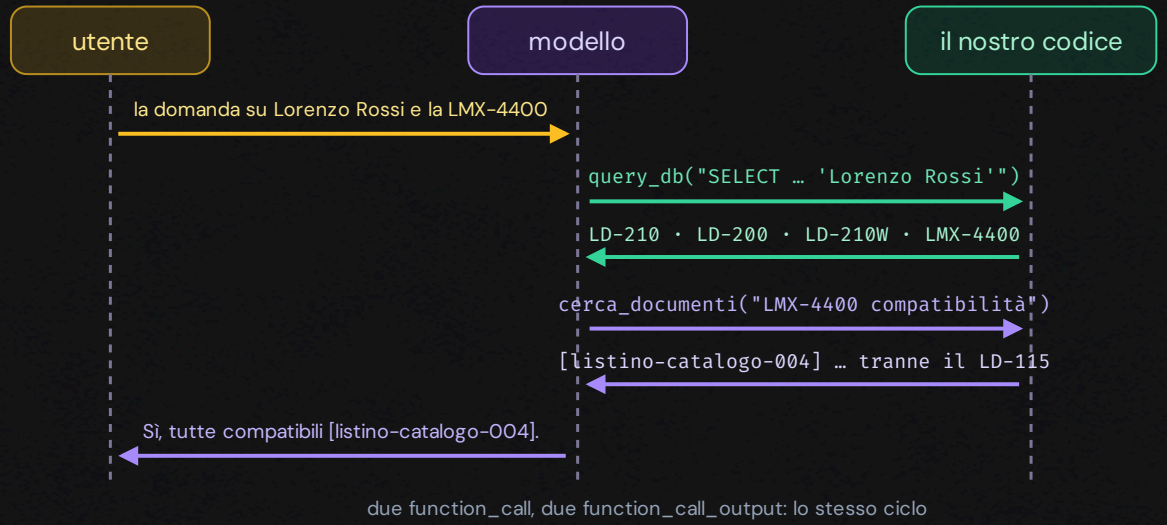


il modello legge domanda + istruzioni + lista dei tool, e chiede quello che gli serve

Non c'è un passo prima che decida la strada: la sceglie il modello dentro il giro, ed è così che la gestisce l'SDK dalla prossima lezione.



Due tool nello stesso giro



due function_call, due function_call_output: lo stesso ciclo

Cosa ha ordinato Lorenzo Rossi sta nel **database**, la compatibilità della LMX-4400 nel **listino**. Il modello chiede due tool: nella stessa risposta, o in due giri di fila.

il ciclo del Lab 1a lo regge già

`strada: documenti+dati`

Se si ferma a `query_db` e risponde con i dati soli, si rilancia: può oscillare.

Quando la risposta non c’è

1 · fuori dai documenti

«Fate sconti agli studenti?»

il retrieval restituisce K chunk lo stesso, **fuori tema**: lo sconto dipendenti della guida, i codici promozionali della policy. Il modello deve accorgersene.

2 · il dato non c'è

«Qual è il numero di telefono di Marco Rossi?»

di solito il modello lo vede dallo schema nella descrizione di `query_db`; se prova la query, riceve come testo `Errore: no such column: telefono`. In tutti e due i casi deve dirlo, non inventare un numero.

3 · fuori da tutto

«Chi ha vinto il campionato di calcio quest'anno?»

nessun tool serve: strada nessuna. La tentazione è rispondere a memoria.

LA RISPOSTA ONESTA

un rifiuto che dice **dove ha cercato**, **senza citazioni**: «Non ho trovato la risposta nei documenti né nei dati.»

IL MODO SBAGLIATO

rispondere a memoria, o citare un chunk che non risponde: una citazione dice «la risposta è qui».

Chi decide è il modello. Chi lo guida: `ISTRUZIONI · TODO 3`



Il log della strada

```
D: Quanti ordini ha fatto Giulia Moretti?
   chiamata 1 … output: reasoning · function_call query_db({"sql":"SELECT COUNT(*) …"})
      eseguo query_db: COUNT(*)
         13
   chiamata 2 … output: message
→ Giulia Moretti ha fatto 13 ordini.
   strada: dati (tool: query_db)

…
RIASSUNTO · K 4 · strada attesa / strada presa
  Entro quanti giorni posso restituire…   documenti      documenti      sì
  Quanti ordini ha fatto Giulia Moretti?   dati           dati           sì
  Lorenzo Rossi può montare il braccio…   documenti+dati documenti+dati sì
  Che tempo fa oggi a Padova?              nessuna        nessuna        sì
  strada giusta: 4/4
```

Sotto ogni risposta, **la strada presa** e i tool in ordine. In fondo, la tabella **attesa / presa**.

«30 giorni [policy-resi-garanzia-001]»
dai documenti, con la citazione: si controlla.

«30 giorni», strada nessuna
a memoria: non si può controllare.

`reasoning` è il ragionamento del modello: non si legge, torna nell'input con il resto. Nel Lab 2 la tabella si guarda prima e dopo ogni leva.



Il file: working/assistente.py

`K — quanti chunk rimanda cerca_documenti (4) · MAX_CHIAMATE — il tetto al giro (6)`

`DOMANDE — quattro, con la strada attesa · LAB2 — True nel Lab 2: legge working/domande_lab2.py · LE_MIE_DOMANDE — le vostre · SOLO — una alla volta`

`TODO 2 (Lab 1b) · CercaDocumenti + cerca_documenti(domanda) — sta sopra perché TOOLS ha bisogno della classe`

`TODO 1 (Lab 1a) · TOOLS + i rami di esegui()`

`TODO 3 (Lab 1c) · ISTRUZIONI`

`chiedi(domanda) — il giro, con dentro il ciclo del TODO 1 (Lab 1a)`

`strada(usati) — documenti / dati / documenti+dati / nessuna`

`helper di stampa · main — ogni giro, poi la tabella attesa / presa`

In cima, importati da `scaffolding/aikit/tools.py` (il vostro `tools.py` della lezione scorsa, promosso nel toolkit): `QueryDb`, `query_db`, `definisci_tool`. K oggi resta 4: non è una leva.

`working/griglia.md — LA SCHEDA DEL LAB 2`

le strade, **gli esiti** e **le due leve**, con la legenda

una tabella: domanda · strada attesa · strada presa · esito · leva · dopo

con cosa chiudo: cosa ho cambiato



Setup

VOI — dalla cartella scripts/ di questa lezione

venv · chiave · primo lancio · 10 minuti · comandi in CONSEGNA.md

1 · AMBIENTE E CHIAVE

Venv nuovo e `pip install -r requirements.txt`: le stesse librerie di Modulo 3 · Lezione 6. La chiave in `scripts/.env`, copiando il `.env` dalla cartella `scripts/` della lezione scorsa. Chi non ce l'ha più alza la mano: il docente ha la chiave.

2 · IL PRIMO LANCIO

`python working/assistente.py`. La prima volta indicizza la collection: una riga per documento, un paio di minuti, poi «indicizzati 82 chunk». Poi risponde da sé alle quattro domande, senza tool: TOOLS è vuota.

FATTO QUANDO

La tabella in fondo dice nessuna su tutte le righe, e sotto `strada giusta: 1/4`: solo Padova coincide, le altre tre hanno una risposta a memoria che la strada non vede. È la baseline, non è rotto.



Lab 1a: attaccare un tool

SCRIVETE VOI — ognuno nel proprio file

`working/assistente.py` · TODO 1 · 45 minuti · consegna in `CONSEGNA.md`

si rientra sulle slide, poi il Lab 1b

Il tool esiste già: `QueryDb` e `query_db` arrivano da `aikit/tools.py`. Manca il cablaggio, in tre punti:

1 · la voce in TOOLS

```
definisci_tool(QueryDb, "query_db")
```

2 · il ramo in esegui()

```
query_db(argomenti["sql"])
```

3 · il ciclo in chiedi(), dove c'è scritto TODO 1

per ogni `function_call`: `json.loads(chiamata.arguments)`, `esegui()`, il nome in `usati`, `stampa_esecuzione()`, e in `storia` la `function_call_output` con lo stesso `call_id` e `str(risultato)`

COME SI PROVA

`SOLO = "Quanti ordini ha fatto Giulia Moretti?"`, in cima al file. Errori tipici: «tool sconosciuto» → manca il ramo; `KeyError: 'sql'` → la chiave non è il campo dello schema; richiede lo stesso tool fino a «mi fermo: 6 chiamate» → la `function_call_output` non torna, o il `call_id` è sbagliato.

FATTO QUANDO

Risponde 13, e sotto la risposta c'è `strada: dati (tool: query_db)`.

Rientro: il ciclo

```
for chiamata in chiamate:                    # può chiederne più d'uno nella stessa risposta
    argomenti = json.loads(chiamata.arguments)   # una stringa JSON → dict
    risultato = esegui(chiamata.name, argomenti)
    usati.append(chiamata.name)              # il log della strada
    stampa_esecuzione(chiamata.name, argomenti, risultato)
    storia.append({
        "type": "function_call_output",
        "call_id": chiamata.call_id,         # lo stesso della richiesta
        "output": str(risultato),            # sempre una stringa
    })
```

Gli argomenti arrivano come stringa JSON, non come dict.

Il call_id è quello della richiesta: altrimenti il modello non sa a cosa stai rispondendo, e richiede lo stesso tool fino al tetto.

L'output è sempre una stringa: i chunk, le righe di una query, un errore.

> Chi non ha finito lo ricopia adesso: il Lab 1b parte da un giro che funziona.



Lab 1b: il RAG come tool

SCRIVETE VOI — ognuno nel proprio file

`working/assistente.py` · TODO 2 · 40 minuti · consegna in `CONSEGNA.md`

si rientra sulle slide, poi la pausa

1 · il tool nuovo

La docstring di `CercaDocumenti`: la descrizione che il modello legge (cosa cerca, in quali documenti, cosa ritorna). E `cerca_documenti(domanda)`: i chunk di `rag.recupera(domanda, K)` come un testo, ognuno con `[id]` davanti. Ritorna una stringa.

2 · attaccarlo, come nel Lab 1a

La seconda voce in `TOOLS`, il secondo ramo in `esegui()`. Il ciclo in `chiedi()` non cambia: vale per qualunque tool.

COME SI PROVA

SOLO sulla domanda dei resi, poi su Lorenzo Rossi. Errori tipici: «ancora da scrivere: TODO 2» → il modello ha chiesto il tool prima della funzione; risposta senza `[id]` → `cerca_documenti` non mette gli id davanti, o sono le istruzioni, che arrivano nel Lab 1c.

FATTO QUANDO

I resi rispondono «30 giorni» citando un id. Lorenzo Rossi ha `strada: documenti+dati`: se esce `dati` da solo, rilancia.



Rientro: cerca_documenti

```python
class CercaDocumenti(BaseModel):                # la docstring è la descrizione che il modello legge
    """Cerca nei documenti aziendali di Lumen (manuale della scrivania LumaDesk,
    policy resi e garanzia, guida allo smart working, listino e catalogo
    prodotti, ticket di supporto, changelog dei rilasci firmware) e ritorna i
    passaggi più pertinenti alla domanda, ognuno preceduto dal suo id da citare.
    I passaggi possono non contenere la risposta."""
    domanda: str


def cerca_documenti(domanda):
    chunks = rag.recupera(domanda, K)           # i K chunk: dict con id, testo, score, source
    testo = ""
    for c in chunks:
        testo += f"[{c['id']}]\n{c['testo']}\n\n"   # l'id davanti: è quello che il modello cita
    return testo.strip()
```

La descrizione elenca i documenti che ci sono: così il modello sa quando cercare qui.

L'ultima frase avvisa che i passaggi possono non contenere la risposta: aiuta il rifiuto.

L'id tra parentesi quadre davanti a ogni chunk è quello che il modello cita.

> Chi non ha finito lo ricopia durante la pausa: il Lab 1c parte da due tool che funzionano.



Pausa

15 minuti

Chi non ha finito il Lab 1b ricopia la slide prima. Al rientro Lab 1c: le istruzioni.

`working/assistente.py · TODO 3`



Lab 1c: le istruzioni

SCRIVETE VOI — ognuno nel proprio file

`working/assistente.py` · TODO 3 · 25 minuti · consegna in CONSEGNA.md

si rientra sulle slide con la tabella

`ISTRUZIONI` · adesso sono quelle di Modulo 3 · Lezione 7. Devono dire:

- chi è e di cosa parla (Lumen)
- **quando usare quale tool**: documenti → `cerca_documenti`, ordini/clienti/prezzi → `query_db`, tutti e due se servono
- cita i passaggi usati con l'id: `[policy-resi-garanzia-001]`
- quando la risposta non c'è: lo dice, e dove ha cercato, senza inventare
- una domanda fuori da Lumen: lo dice, senza usare i tool

da `rag.ISTRUZIONI` (in `scaffolding/aikit/rag.py`) si riusa la citazione con l'id; il resto parla solo di documenti e va riscritto per due fonti

`COME SI PROVA`

`SOLO = None`: tutte le `DOMANDE` e la tabella. Una riga dice NO? Rilanciala in `SOLO` due volte: se almeno una volta passa a sì, conta come fatta (il modello oscilla); se resta NO tutte e due le volte, è una cosa da istruzioni.

`FATTO QUANDO`

La tabella dice **4/4**, e Padova viene rifiutata senza chiamare tool.



Rientro: le istruzioni della soluzione

```
Sei l'assistente di Lumen, un'azienda che vende scrivanie regolabili LumaDesk
e accessori. Rispondi in italiano, in poche frasi.

Per rispondere hai due strade:
- domande su prodotti, procedure, resi, garanzia, smart working, ticket e
rilasci firmware: usa cerca_documenti e cita i passaggi che usi con il loro id
tra parentesi quadre, per esempio [policy-resi-garanzia-001];
- domande su ordini, clienti e prezzi: usa query_db.
Se una domanda ha bisogno di tutte e due, usa tutti e due i tool.

Rispondi SOLO con quello che i tool ti restituiscono. Se i passaggi trovati
non contengono la risposta, o i dati non ce l'hanno, di' che non hai trovato
la risposta e dove hai cercato. Se una domanda non riguarda Lumen, di' che non
puoi rispondere, senza usare i tool. Se un tool ritorna un errore due volte di
fila, fermati e di' che il dato non è disponibile. Non inventare mai.
```

`viola` · quando usare quale tool

`verde` · come citare

`giallo` · quando la risposta non c'è

Le vostre possono essere diverse: contano queste tre cose. Chi non è a 4/4 ricopia queste prima del Lab 2.



Lab 2: provare e sistemare

SCRIVETE VOI — le domande e le leve, non le funzioni

`working/assistente.py` · `LAB2 = True` · 45 minuti · si annota in `working/griglia.md`

si rientra con la griglia

`1 · LE VOSTRE DOMANDE DI MODULO 3 · LEZIONE 6`

Le nove che il RAG da solo sbagliava, più una doppia: sono in `working/domande_lab2.py`, con la strada attesa. In cima al file `LAB2 = True`, un lancio, poi una riga in griglia per domanda: strada presa, **quante volte ha cercato e con che parole** (nel log: `cerca_documenti → cerca_documenti`), esito. La risposta vera: nei ticket e nel changelog in `dataset/lumen_m3/`, o con `scaffolding/sql.py`.

`2 · SISTEMARE, UNA LEVA ALLA VOLTA`

Per ogni domanda sbagliata: `ISTRUZIONI` (per esempio «se i passaggi non contengono la risposta, cerca di nuovo con altre parole»; «se è un ticket, cita il codice TCK») o la docstring di `CercaDocumenti` (dire che ci sono i ticket con i casi risolti). Si rilancia su **tutte** le domande e si scrive «dopo», comprese quelle che erano giuste e ora no (prima di dirlo: SOLO e due rilanci). Chi finisce: domande proprie.

gli esiti: giusta | strada sbagliata | risposta sbagliata | inventata (anche: cita un chunk che non c'entra)

`FATTO QUANDO`

Almeno **due domande sbagliate** sono sistemate senza romperne altre.

Le due leve a confronto

leva	cosa muove	il rischio
`ISTRUZIONI`	la scelta del tool, quante volte cerca e con che parole, le citazioni, il rifiuto	troppo rigide: rifiuta anche quando la risposta c'è; «cerca ancora» senza un tetto: giri lunghi
`la docstring di CercaDocumenti`	se e quando il modello sceglie i documenti, e cosa sa che c'è dentro	troppo larga: cerca nei documenti anche i dati

Di solito basta la prima. La seconda serve quando il modello non entra nei documenti, o non sa cosa c'è dentro.

Rientro finale, dentro i 10'. Alla lavagna, chi vuole:

la domanda che ha ingannato più persone | la leva che l'ha curata | una leva che ha rotto qualcos'altro



Recap

1. Un tool si attacca in **tre punti**: la voce in `TOOLS`, il ramo in `esegui()`, e il ciclo che esegue le `function_call` e rimanda i risultati con lo stesso `call_id`.

2. Il RAG è un **tool come gli altri**: `cerca_documenti` restituisce i chunk con gli id, e le regole stanno nelle `ISTRUZIONI`.

3. Il modello **sceglie la strada** da istruzioni e descrizioni, dentro il giro; la strada presa si stampa sempre.

4. Quando nessuna strada ha la risposta, il **rifiuto** dice dove ha cercato, senza citare.



La prossima lezione

oggi
il giro scritto a mano, lo stesso di Modulo 3 · Lezione 7, con il ciclo che avete scritto voi

→

Modulo 3 · Lezione 10: il giro gestito dall'OpenAI Agents SDK
il giro lo gestisce la libreria, con questi stessi due tool; ogni pezzo dell'SDK corrisponde a una riga che avete già scritto

`cerca_documenti` `query_db` restano uguali



Grazie

prossima lezione: Modulo 3 · Lezione 10

